

Domain-Driven Design (DDD) e Arquitetura de Softwares Escaláveis com Java

Atividade Prática — Agregados, Consistência Transacional e Eventos de Domínio

Aluno: Lucas Amorim Porciuncula
Instituto Infnet
Professor: Leonardo Silva da Gloria

1. Explique de forma sucinta qual a razão de criar Agregados.

Um Agregado existe para transformar um conjunto de objetos relacionados (entidades e Value Objects) em uma única unidade de consistência, com um único ponto de entrada: a Raiz do Agregado (Aggregate Root). Sem essa fronteira, as regras de negócio ficam espalhadas por vários services, e nada impede que um objeto seja alterado para um estado inválido por um caminho que o service "esqueceu" de validar.

No `payment-service`, é o agregado `Pagamento` quem decide, sozinho, se pode nascer ou mudar de status — ninguém de fora manipula seus atributos diretamente. Isso concentra a regra de negócio em um único lugar, define claramente até onde vai uma transação e facilita testar, evoluir e até escalar/particionar o sistema por agregado.

2. O que significa "consistência transacional"?

É a garantia de que, ao final de uma transação, o estado persistido está integralmente correto em relação às regras de negócio — ou a operação é aplicada por completo, ou não é aplicada de forma alguma. Não existe meio-termo, como um `Pagamento` salvo com status `CONFIRMADO` mas com o valor de outra transação que falhou.

Em DDD, a prática recomendada é que **uma transação de banco de dados altere no máximo um Agregado por vez**. Isso simplifica o controle de concorrência (lock no nível do agregado) e evita que a consistência de um objeto dependa do sucesso de uma escrita em outro, o que se torna essencial quando o sistema cresce e os agregados passam a viver em serviços e bancos diferentes — como já é o caso entre `payment-service` e o monólito legado.

3. Cite as 4 propriedades cruciais que definem transações.

São as propriedades ACID:

Propriedade	O que garante
Atomicidade	A transação é tudo ou nada — se uma parte falha, tudo é desfeito (rollback).
Consistência	A transação leva o banco de um estado válido para outro estado válido, respeitando as regras/invariantes.
Isolamento	Transações concorrentes não enxergam estados intermediários umas das outras.
Durabilidade	Uma vez confirmada (commit), a alteração sobrevive a falhas (fica gravada em disco).

4. O que são "invariantes de negócio"?

São regras que precisam ser verdadeiras durante **todo** o ciclo de vida de um objeto de domínio, não apenas no momento da criação. Diferente de uma validação de formulário (que checa uma entrada pontual), uma invariante é uma condição estrutural do próprio modelo: se ela for quebrada em qualquer instante, o objeto deixa de fazer sentido para o negócio.

No agregado `Pagamento`, duas invariantes são: valor sempre positivo e menor ou igual a R\$ 10.000,00, e cartão terminado em "0000" nunca pode gerar um pagamento aprovado. Essas regras vivem dentro do próprio construtor/fábrica do agregado — é fisicamente impossível instanciar um `Pagamento` em memória que as viole.

5. Por que um agregado só deve ter acesso a outro agregado pelo ID?

Porque cada agregado é sua própria fronteira transacional. Se um agregado guardasse uma referência de objeto completo para outro (em vez do ID), a tendência natural seria alterar os dois dentro da mesma transação — o que quebra a regra "uma transação, um agregado" e obriga um lock conjunto, prejudicando concorrência e escalabilidade.

Referenciar só pelo ID também evita carregar grafos de objetos gigantes na memória sem necessidade, remove acoplamento direto entre os módulos (cada agregado pode evoluir seu próprio schema sem quebrar o outro) e é o que torna possível, no limite, que os dois agregados morem em bounded contexts ou serviços físicos diferentes — exatamente o caso do `payment-service`: ele nunca carrega o agregado `Pedido` inteiro, só guarda um `PedidoId`, porque `Pedido` pertence ao monólito legado, outro bounded context.

6. Entidade que representa um agregado referenciando outro agregado pelo ID.

Crie um trecho de código Java de uma entidade que represente um agregado e faça referência a outro agregado dentro do escopo do

projeto.

O agregado `Pagamento` referencia o agregado `Pedido` (que pertence ao monólito) exclusivamente através do Value Object `PedidoId` — nunca da entidade `Pedido` inteira:

```
public class Pagamento {

    private final PagamentoId id;
    private final PedidoId pedidoId;      // referência por ID ao agregado Pedido
    private final Valor valor;
    private final NumeroCartao numeroCartao;
    private StatusPagamento status;

    public static Pagamento criar(PagamentoId id, PedidoId pedidoId,
                                   Valor valor, NumeroCartao numeroCartao) {
        if (valor.getQuantia().compareTo(LIMITE_MAXIMO) > 0) {
            throw new PagamentoRecusadoException("Valor acima do limite permitido");
        }
        if (numeroCartao.estaBloqueado()) {
            throw new PagamentoRecusadoException("Cartão bloqueado");
        }
        return new Pagamento(id, pedidoId, valor, numeroCartao, StatusPagamento.PENDENTE);
    }
    // getters omitidos
}
```

Código completo: [Pagamento.java](#) e [PedidoId.java](#) — repositório *payment-service*, branch *main*.

7. Método de negócio com previsão de publicação de um evento de domínio.

Elabore um trecho de código Java que mostre um método de negócio com a previsão de publicação de um evento de domínio.

O método `confirmar()` do agregado `Pagamento` muda o status e, no mesmo passo, registra o evento de domínio `PagamentoConfirmadoEvent` em uma lista interna. O agregado não publica o evento sozinho (isso seria responsabilidade de infraestrutura) — ele só o disponibiliza para a camada de aplicação:

```
public class Pagamento {

    private final List<DomainEvent> eventos = new ArrayList<>();

    public void confirmar() {
        if (this.status == StatusPagamento.CONFIRMADO) {
            throw new IllegalStateException("Pagamento já confirmado");
        }
        this.status = StatusPagamento.CONFIRMADO;

        this.eventos.add(new PagamentoConfirmadoEvent(
            this.id, this.pedidoId, this.valor.getQuantia(), Instant.now()
        ));
    }

    public List<DomainEvent> getEventos() {
        return Collections.unmodifiableList(eventos);
    }

    public void limparEventos() {
        this.eventos.clear();
    }
}
```

Quem lê `getEventos()` e publica de fato é a camada de aplicação (`PagamentoApplicationService`), depois que a transação de persistência foi confirmada — assim o evento só sai para o mundo se o `Pagamento` realmente foi salvo.

Implementação completa: método `confirmar()` em [Pagamento.java](#), repositório *payment-service*.

8. O que é "evento de domínio"?

É um fato imutável, já ocorrido dentro do domínio, que é relevante o suficiente para ser comunicado a outras partes do sistema — outros agregados, outros bounded contexts ou sistemas externos. A ideia central é: o evento sempre descreve o **passado** ("o pagamento FOI confirmado"), nunca uma ordem ou intenção ("confirme o pagamento").

Ele carrega os dados relevantes do momento em que aconteceu (quem, quando, quanto) e serve para desacoplar quem gerou o fato de quem precisa reagir a ele: o *payment-service* não precisa saber que o monólito vai atualizar o status do `Pedido`, só precisa publicar que o pagamento foi confirmado.

9. Abstração de um objeto do tipo "evento de domínio".

Crie um trecho de código Java que mostre uma abstração de um objeto do tipo "evento de domínio".

```
package com.exemplo.paymentservice.domain;

import java.time.Instant;

/**
 * Abstração de um evento de domínio: um fato imutável que já
 * aconteceu dentro de um agregado.
 */
public interface DomainEvent {

    Instant ocorridoEm();
}
```

Código no repositório: [DomainEvent.java](#) — repositório *payment-service*.

10. Implementação de um "evento de domínio".

Crie um trecho de código Java que mostre a implementação de um "evento de domínio".

```
package com.exemplo.paymentservice.domain;

import java.math.BigDecimal;
import java.time.Instant;

/**
 * Evento concreto: "um pagamento foi confirmado".
 * Usado pelo monólito legado para atualizar o status do Pedido
 * sem precisar conhecer o agregado Pagamento por inteiro.
 */
public record PagamentoConfirmadoEvent(
    PagamentoId pagamentoId,
    PedidoId pedidoId,
    BigDecimal valorConfirmado,
    Instant ocorridoEm
) implements DomainEvent {
}
```

Código no repositório: [PagamentoConfirmadoEvent.java](#) — repositório *payment-service*.

11. Diferença entre filas e tópicos, e como funcionam em conjunto.

Fila (queue): comunicação ponto-a-ponto. Uma mensagem colocada na fila é entregue a **um único** consumidor entre os que estão competindo por ela (competing consumers) — assim que consumida, some da fila. É o modelo ideal para distribuir trabalho (ex.: processar boletos) sem duplicar a execução.

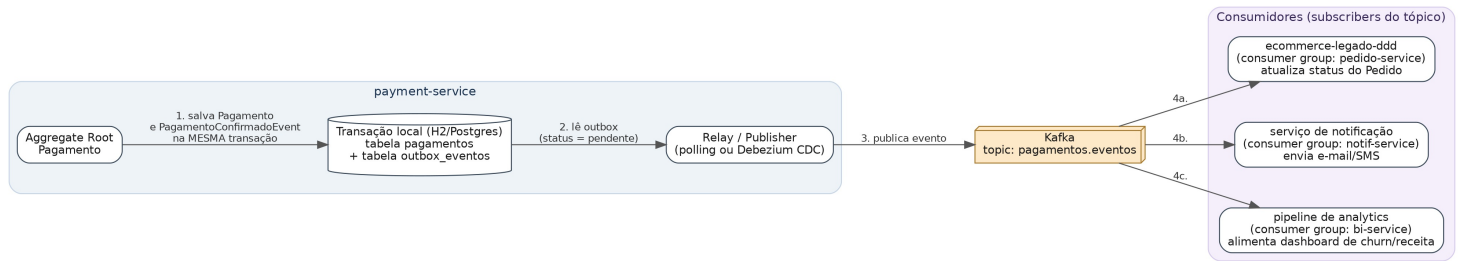
Tópico (topic / publish-subscribe): comunicação de um-para-muitos. Uma mensagem publicada no tópico é entregue a **todos** os assinantes (ou a todos os grupos de consumidores) independentemente uns dos outros — cada um lê sua própria cópia lógica do fluxo.

Eles funcionam em conjunto em arquiteturas orientadas a eventos, como o Kafka: o evento é publicado uma única vez em um tópico (ex.: `pagamentos.eventos`), e por trás, cada *consumer group* inscrito recebe sua própria cópia — o serviço do monólito que atualiza o Pedido, o serviço de notificação e o pipeline de analytics podem consumir o mesmo evento em paralelo, cada um na sua própria "fila" lógica dentro do tópico, sem que um interfira no outro.

12. Exemplo de arquitetura para publicação de eventos de domínio.

Dê um exemplo de solução de arquitetura para publicação de eventos de domínio dentro do escopo do projeto (ideal um desenho).

A solução usa o padrão **Transactional Outbox** combinado com um broker de eventos (Kafka), evitando o problema clássico de "salvei o Pagamento no banco, mas caí antes de publicar o evento" (ou o inverso: publiquei o evento mas a transação deu rollback).



Fluxo: o agregado gera o evento em memória → application service salva Pagamento + evento na tabela outbox na MESMA transação → um relay assíncrono lê a outbox e publica no tópico Kafka → cada consumidor (monólito, notificação, analytics) processa de forma independente.

O ponto-chave é o passo 1: o registro na tabela `outbox_eventos` acontece dentro da mesma transação ACID que salva o `Pagamento`. Isso garante atomicidade entre "gravar o estado" e "garantir que o evento será publicado" — se a transação falha, nenhum dos dois acontece; se der certo, o relay eventualmente vai publicar o evento (at-least-once), mesmo que o processo caia entre os passos 1 e 3.

13. Finalidade de uma Event Store.

Uma Event Store é um banco de dados especializado em armazenar, de forma append-only (só se adiciona, nunca se sobrescreve ou apaga), a sequência completa de eventos de domínio que já ocorreram. Ela é a fonte da verdade (source of truth) do sistema — em vez de guardar "o estado atual", guarda "tudo que aconteceu para chegar até aqui".

Isso serve para: reconstruir o estado de qualquer agregado a partir do zero (replay), auditar com precisão total o que aconteceu e quando (essencial em domínios financeiros, como pagamentos), alimentar novas projeções/read models sem tocar no domínio original, e depurar problemas de produção reproduzindo exatamente a sequência de eventos que levou a um estado inesperado.

14. Event Sourcing vs. persistência tradicional.

O que é Event Sourcing e como ele se diferencia da persistência tradicional em bancos de dados relacionais? Explique como os eventos salvos são usados para recuperar o estado atual de um Agregado.

Na persistência tradicional (a que o `payment-service` usa hoje via JPA), cada agregado corresponde a uma linha em uma tabela que é **sobrescrita** a cada mudança de estado: quando `Pagamento.confirmar()` é chamado, o `UPDATE` troca `status = 'PENDENTE'` por `status = 'CONFIRMADO'`, e o valor anterior se perde.

Em Event Sourcing, em vez de salvar o estado atual, salva-se a **sequência de eventos** que representa cada mudança: `PagamentoCriado`, `PagamentoConfirmado`, `PagamentoEstornado` — cada um como um registro imutável e append-only. O estado "atual" nunca é gravado diretamente; ele é sempre derivado.

Para recuperar o estado atual de um agregado, o sistema busca **todos** os eventos daquele agregado (filtrando pelo ID, ex.: `PagamentoId`) ordenados cronologicamente, cria uma instância vazia do agregado e "reaplica" cada evento em sequência (replay), um método `aplicar(evento)` por tipo de evento, até reconstituir o estado corrente — exatamente como se o agregado tivesse acabado de vivenciar sua própria história de novo. Em agregados com histórico muito longo, usa-se *snapshots* periódicos do estado para não precisar reprocessar milhares de eventos a cada leitura.